

PMGI System - Complete Data Flow Documentation

System Overview

The PMGI (Performance Management Growth Initiative) system is a comprehensive Laravel-based application designed to manage officer performance evaluations, appointments, and committee operations.

User Authentication Flow

User Login → fms_users Table → Role Check → Dashboard Access

User Roles:

- **PYD**: Performance review and data analysis
- **PYM**: Officer appointment management
- **PMC**: Committee setup and management
- **HR**: Human resource data management
- **JTT**: Joint training team coordination

PYD Performance Data Flow

Database Tables:

1. **pmgi_summ_mth_officer** - Monthly performance data
2. **pmgi_bank_officer** - Officer master information
3. **pmgi_hrd_officer** - HR officer details

Process Flow:

1. PYD officer selects date range
2. System queries pmgi_summ_mth_officer table
3. Joins with pmgi_bank_officer for officer details
4. Filters by incl_pmgi_flag (excludes 'G', 'H')
5. Displays cumulative performance or "No Data" dialog

SQL Query:

```
SELECT * FROM pmgi_summ_mth_officer s
JOIN pmgi_bank_officer b ON s.officer_id = b.officer_id
WHERE s.officer_id = 'OFFICER_ID'
      AND s.report_date BETWEEN 'start_date' AND 'end_date'
      AND s.incl_pmgi_flag NOT IN ('G', 'H')
ORDER BY s.report_date ASC;
```

PYM Appointment Data Flow

Database Tables:

1. **pmgi_session_info** - General session information
2. **pmgi_session_pym_info** - PYM appointment details
3. **pmgi_bank_officer** - Officer information

Process Flow:

1. Create new session in pmgi_session_info
2. Save appointment details in pmgi_session_pym_info
3. Load email template (lantikan_pym_pmc.blade.php)
4. Replace placeholders with officer data
5. Convert HTML to PNG using wkhtmltoimage

6. Send email with PNG attachment
7. Update session status and queue cleanup

Email Generation:

- **Template:** lantikan_pym_pmc.blade.php
- **Logo:** public_path('image/logo-tekun.png')
- **Fonts:** System fonts (Arial, Helvetica, sans-serif)
- **Storage:** storage/app/public/emails/pym_pmc/

PMC Committee Data Flow

Database Tables:

1. **pmgi_session_info** - Session management
2. **pmgi_session_pmc_info** - Committee details
3. **pmgi_sett_state_committee** - State settings

State Code Formatting:

- Input: 2, 3, 5, 10, 12
- Output: 02, 03, 05, 10, 12 (zero-padded)
- Implementation: `str_pad($stateCode, 2, '0', STR_PAD_LEFT)`

Process Flow:

1. Create PMC session
2. Format state codes with zero-padding
3. Save to pmgi_sett_state_committee
4. Generate committee invitations
5. Send emails to multiple recipients

Session Monitoring

pmgi_mntr_session Table:

- **pmgi_result values:** CP1, CP2, PEX, EXC, EXP, PDQ, EXL
- **Null safety:** All relationship access protected
- **Template checks:**

```
@if($data->mntrSession && $data->mntrSession->pmgi_result == 'CP1')
```

Email Conversion Process

wkhtmltoimage Parameters:

- `--page-size A4`
- `--orientation Portrait`
- `--javascript-delay 500`
- `--no-stop-slow-scripts`
- `--load-error-handling ignore`
- `--disable-external-links`
- `--enable-local-file-access`
- `--disable-smart-width`

Error Handling:

1. **Primary:** Direct wkhtmltoimage via `proc_open()`
2. **Timeout:** 30-second limit with force kill
3. **Fallback:** Laravel Artisan command
4. **Logging:** Comprehensive stdout/stderr capture

File Management

Directory Structure:

```
storage/app/public/emails/  
├─ pym_pmc/  
│   └─ email_pym_pmc_OFFICER_ID.png  
└─ urusetia_negeri/  
    └─ email_urusetia_negeri_OFFICER_ID.png
```

Cleanup Process:

- **Job:** CleanupTemporaryFiles
- **Schedule:** Files older than 24 hours
- **Safety:** array_filter() for null values
- **Types:** PNG and HTML files

Database Relationships

Core Relationships:

```
fms_users → pmgi_session_info (creates) pmgi_session_info →  
pmgi_session_pym_info (contains) pmgi_session_info →  
pmgi_session_pmc_info (contains) pmgi_bank_officer →  
pmgi_summ_mth_officer (has performance) pmgi_bank_officer →  
pmgi_session_pym_info (appointed to) pmgi_session_pmc_info →  
pmgi_sett_state_committee (belongs to)
```

Compatibility Fixes

SQL Server Compatibility:

- **Before:** `substr(b.branch_code, 0, 2)`
- **After:** `SUBSTRING(b.branch_code, 1, 2)`
- **Reason:** SQL Server uses 1-based indexing

Path Handling:

- **Cross-platform:** Uses DIRECTORY_SEPARATOR constant
- **Mixed separators:** Normalized Windows/Linux paths
- **Trim handling:** Prevents double separators

Font Issues:

- **Removed:** Google Fonts external links
- **Replaced:** System fonts for reliability
- **Fixed:** External image loading causing hangs

Error Handling

Null Safety:

- All relationship access protected with null checks
- Template rendering secured against null objects
- Database queries handle missing relationships

Process Management:

- 30-second timeout enforcement
- Stream monitoring for hang detection
- Graceful process termination
- Comprehensive error logging

System Performance

Database Optimization:

- Indexed searches on officer_id, report_date
- Efficient JOIN operations
- Filtered queries to reduce data load

Resource Management:

- Background job processing
- Automatic file cleanup
- Memory-efficient stream handling
- Process timeout prevention

Note: This documentation covers the complete data flow and technical implementation of the PMGI system, including all the fixes and optimizations implemented during development.